

발표 보충 자료 — 이 AWS 서비스들이 뭔가요

대규모 시스템 설계 스터디 4주차 · 6·7장 발표 부록 | 송승환

발표에서 책 개념을 AWS 서비스와 연결하는 장면이 여러 번 나옵니다. 그런데 그 서비스들 자체를 모르면 연결이 와 닿지 않습니다. 그래서 발표 중에 이름만 스쳐 지나가는 서비스들이 각각 뭔지를 따로 정리했습니다.

발표를 따라오는 데 이 문서를 미리 읽을 필요는 없습니다. 발표 중에 모르는 이름이 나오면 해당 절을 펴 보시고, 끝나고 나서 천천히 읽으셔도 됩니다.

01 먼저 — 리전과 가용 영역

AWS의 거의 모든 이야기가 이 두 단어 위에서 돌아갑니다.

용어	뜻
리전 (Region)	지리적으로 떨어진 큰 단위. 서울 리전은 ap-northeast-2. 리전이 다르면 물리적 거리가 멀어 통신 지연이 큼니다.
가용 영역 (AZ)	한 리전 안에 있는, 물리적으로 분리된 데이터센터 묶음. 서울 리전에는 네 개가 있습니다. 전원·냉방·네트워크가 서로 독립적이라 한 곳이 통째로 죽어도 나머지는 살아 있습니다. AZ끼리는 전용망으로 이어져 있어 지연이 아주 작습니다.

왜 이게 중요한가

"여러 AZ에 걸쳐 둔다"는 말은 같은 것을 여러 건물에 나눠 보관한다는 뜻입니다. 뒤에 나오는 Multi-AZ, Aurora의 6벌 저장기 전부 이 아이디어 위에 있습니다.

02 트래픽을 나누는 장치 — ELB

로드밸런서(Load Balancer)는 들어오는 요청을 여러 서버에 나눠 보내는 장치입니다. 하는 일은 셋입니다.

- 요청을 서버들에 분배한다
- 주기적으로 상태를 확인해(Health Check) 죽은 서버는 제외한다
- 서버가 늘고 줄어도 클라이언트는 모르게 한다

AWS의 관리형 로드밸런서를 통틀어 ELB(Elastic Load Balancing)라고 부르고, 종류가 넷입니다. 이 중 CLB는 구세대라 새로 쓰지 않습니다.

계층(Layer) 이야기 — 셋을 가르는 기준

네트워크 통신은 여러 계층으로 나뉘는데, 로드밸런서를 고를 때는 어느 계층까지 들여다보느냐가 기준이 됩니다.

- L3(네트워크 계층) — IP 패킷 수준. 내용은 거의 안 봅니다.
- L4(전송 계층) — TCP/UDP 포트 수준. "어느 포트로 가는 연결인가"까지만 봅니다.
- L7(응용 계층) — HTTP 수준. URL 경로, 헤더, 쿠키까지 들여다봅니다.

위로 갈수록 세밀하게 판단할 수 있지만 그만큼 처리가 무겁고, 아래로 갈수록 내용을 안 보고 넘겨서 빠릅니다. 이 맞바꿈이 ALB와 NLB의 성격 차이를 만듭니다.

ALB — Application Load Balancer (L7)

- HTTP / HTTPS / WebSocket 트래픽을 다루고, HTTP/2를 지원합니다.
- 요청 내용을 보고 어디로 보낼지 고를 수 있습니다. 경로(/users와 /posts), 호스트 이름(one.example.com과 other.example.com), 쿼리 스트링, 헤더 기준으로 서로 다른 **대상 그룹(Target Group)**에 보냅니다.
- 그래서 마이크로서비스나 컨테이너 환경에 잘 맞습니다. 하나의 ALB가 여러 서비스를 앞에서 받아 갈라줍니다.
- HTTP에서 HTTPS로 리다이렉트하는 기능이 내장돼 있습니다.
- 고정된 호스트 이름(XXX.region.elb.amazonaws.com)을 받습니다. IP는 고정 아닙니다.

알아두면 좋은 함정

ALB를 거치면 뒤에 있는 서버는 **클라이언트의 진짜 IP를 볼 수 없습니다**. ALB가 연결을 끊고 새로 맺기 때문에 서버에는 ALB의 IP가 찍힙니다. 원래 IP는 X-Forwarded-For 헤더에 담겨 오므로, 접속 IP를 기록하거나 차단하는 로직이 있다면 이 헤더를 읽어야 합니다. (포트는 X-Forwarded-Port, 프로토콜은 X-Forwarded-Proto)

NLB — Network Load Balancer (L4)

- TCP / UDP / TLS 트래픽을 그대로 넘깁니다. HTTP가 아니어도 됩니다.
- 초당 수백만 건을 처리하고 지연이 극도로 낮습니다. 내용을 안 보고 넘기기 때문입니다.
- **AZ당 고정 IP를 하나씩** 가지며, 탄력적 IP를 직접 붙일 수도 있습니다. 상대 회사가 "특정 IP만 허용" 정책을 쓸 때 이게 필요합니다.
- 대상으로 EC2 인스턴스, 사설 IP, 심지어 ALB를 둘 수 있습니다. (NLB 뒤에 ALB를 두는 구성)

GWLB — Gateway Load Balancer (L3)

- 앞의 둘과 목적이 다릅니다. 웹 트래픽을 나누는 게 아니라, **방화벽·침입 탐지(IDS/IPS)·패킷 검사** 같은 보안 장비를 트래픽 경로에 끼워 넣기 위한 것입니다.
- 모든 트래픽이 지나가는 단일 관문 역할과, 그 트래픽을 여러 보안 장비에 나눠주는 역할을 함께 합니다.
- IP 패킷 수준(L3)에서 동작하며, GENEVE 프로토콜 6081 포트를 씁니다.

정리 — 언제 무엇을

종류	계층	다루는 트래픽	이럴 때 씁니다
ALB	L7	HTTP · HTTPS · WebSocket	웹 서비스, REST API, 마이크로서비스 — 대부분의 경우
NLB	L4	TCP · UDP · TLS	극한의 성능이 필요하거나, HTTP가 아닌 프로토콜이거나, 고정 IP가 필요할 때
GWLB	L3	IP 패킷	방화벽 등 보안 장비를 트래픽 경로에 넣을 때
CLB	L4/L7	—	구세대. 새로 만들 이유가 없습니다.

ASG도 같이 나오는 이유

Auto Scaling Group(ASG)은 부하에 따라 EC2 대수를 자동으로 늘리고 줄이는 기능입니다. 로드밸런서와 짝으로 쓰입니다 — ASG가 서버를 늘리면 LB가 새 서버에도 트래픽을 보내고, 줄이면 빼는 식입니다. 둘이 같이 있어야 "알아서 늘어나는 시스템"이 완성됩니다.

03 DB를 여러 벌 두는 법 — Multi-AZ와 Read Replica

RDS에서 DB를 여러 벌 두는 방법이 둘인데, **목적이 완전히 다릅니다**. 이름이 비슷해서 자주 헷갈립니다.

Multi-AZ — 장애에 대비하는 것

- 다른 AZ에 대기(**standby**) 인스턴스를 하나 두고, 원본에 쓰는 내용을 동기(**synchronous**)로 그대로 복사합니다.
- 동기라는 건, 대기 쪽까지 기록이 끝나야 쓰기가 완료된다는 뜻입니다. 그래서 둘의 내용이 항상 같습니다.
- DNS 이름이 하나라서, 원본이 죽으면 대기 인스턴스가 자동으로 승격되고 **애플리케이션은 아무것도 바꿀 필요가 없습니다**.
- AZ 하나가 통째로 죽거나, 인스턴스·스토리지·네트워크가 고장 나는 상황을 대비합니다.

가장 많이 하는 오해 두 가지

- ① **Multi-AZ는 읽기 분산이 아닙니다**. 대기 인스턴스는 평소에 아무 요청도 받지 않습니다. 놓고 있다가 원본이 죽을 때만 올라옵니다. 읽기를 나누고 싶으면 Read Replica를 따로 붙여야 합니다.
- ② **Multi-AZ는 샤딩이 아닙니다**. 이름에 Multi가 붙어서 "여러 개로 쪼갬"으로 읽히지만, 쪼개는 게 아니라 **같은 데이터를 통째로 한 벌 더 두는 것**입니다. 복제이지 분할이 아닙니다.

Read Replica — 읽기 부하를 나누는 것

- 읽기 전용 복제본을 최대 15대까지 둘 수 있습니다. 같은 AZ, 다른 AZ, 다른 리전 어디든 가능합니다.
- 복제가 비동기(**asynchronous**)입니다. 원본에 쓰고 나서 복제본에 전달되기까지 시간 차가 있습니다.
- 그래서 방금 쓴 데이터가 복제본에서는 아직 안 보일 수 있습니다. 이 시간 차를 복제 지연(replication lag)이라고 하고, 이 상태가 바로 6장에서 말하는 **결과적 일관성(eventual consistency)**입니다.
- 복제본에는 SELECT만 보낼 수 있습니다. INSERT·UPDATE·DELETE는 안 됩니다.
- 애플리케이션이 읽기용 연결 문자열을 따로 써야 합니다. Multi-AZ와 달리 저절로 되지 않습니다.
- 필요하면 복제본을 독립된 DB로 승격시킬 수 있습니다.

	Multi-AZ	Read Replica
목적	장애 대비 (가용성)	읽기 부하 분산 (확장)
복제 방식	동기 (synchronous)	비동기 (asynchronous)
일관성	강한 일관성	결과적 일관성
평소 역할	없음 — 대기만 함	읽기 요청을 처리함
앱 수정	필요 없음	읽기용 연결을 따로 써야 함
6장 대응	데이터 다중화 · "데이터 센터 장애 처리"	데이터 다중화 — 읽기 확장이 목적

비용 한 줄

AWS는 보통 AZ 사이로 데이터가 오갈 때 전송 요금을 받습니다. 다만 같은 리전 안의 RDS Read Replica 복제는 예외로 이 요금이 없습니다. 리전을 넘어서면 과금됩니다.

04 리전을 넘어서 — Cross-Region

지금까지는 한 리전 안의 이야기였습니다. 리전 자체가 문제가 되는 경우가 둘 있습니다 — 지역 전체 규모의 재해, 그리고 멀리 있는 사용자의 지연입니다.

Cross-Region Read Replica

- Read Replica를 아예 다른 리전에 두는 것입니다.
- 서울 리전에 통째로 문제가 생겨도 도쿄의 복제본을 승격시켜 서비스를 이어갈 수 있습니다.
- 유럽 사용자가 많다면 유럽 리전에 복제본을 두어 읽기 지연을 줄일 수도 있습니다.
- 리전 간 전송이므로 네트워크 비용이 붙습니다.

Aurora Global Database

Aurora를 쓴다면 이쪽이 권장되는 방법입니다.

- 주 리전 한 곳이 읽기·쓰기를 담당하고, 보조 리전을 최대 10곳까지 읽기 전용으로 둡니다.
- 리전 간 복제 지연이 보통 1초 미만입니다.
- 보조 리전 하나당 읽기 복제본을 최대 16대까지 둘 수 있습니다.
- 재해가 나면 보조 리전을 주 리전으로 승격하는 데 1분 미만이 걸립니다.

05 Aurora — 발표에서 가장 중요한 서비스

Aurora는 AWS가 직접 만든 데이터베이스입니다. MySQL·PostgreSQL과 호환되도록 만들어져서, 기존 드라이버와 쿼리를 그대로 쓸 수 있습니다. 애플리케이션 입장에서는 그냥 MySQL에 연결하는 것과 같습니다.

왜 쓰나

- AWS는 RDS의 MySQL 대비 약 5배, PostgreSQL 대비 약 3배 성능이라고 밝히고 있습니다.
- 스토리지가 10GB 단위로 자동으로 늘어납니다. 용량을 미리 잡아둘 필요가 없습니다. (상한은 AWS가 계속 올리고 있어 최신 문서 확인을 권장합니다.)
- 읽기 복제본을 최대 15대까지 둘 수 있고, 복제 지연이 10밀리초 미만으로 RDS보다 훨씬 짧습니다.
- 장애 치료가 30초 미만으로 끝납니다. 고가용성이 기본 설계에 들어 있습니다.
- 연결 지점이 둘로 나뉩니다 — **Writer Endpoint**는 쓰기를 담당하는 마스터를 가리키고, **Reader Endpoint**는 읽기 복제본들에 부하를 자동으로 분산해 줍니다. 복제본이 늘고 줄어도 애플리케이션은 이 주소만 알면 됩니다.

6장과 직접 연결되는 부분

Aurora의 저장 구조가 책 6장의 내용 그 자체입니다.

데이터를 3개 AZ에 6벌 저장하고

쓰기 — 6개 중 4개의 응답을 기다림 $W = 4$
읽기 — 6개 중 3개의 응답을 기다림 $R = 3$

$N = 6, W = 4, R = 3$
 $W + R = 7 > N = 6$ ← 책의 정족수 조건

책에서 배운 $W + R > N$ 이면 강한 일관성이 보장된다는 공식이, Aurora에서 실제로 이 숫자로 돌아가고 있습니다. 쓴 4개와 읽은 3개는 6개 중에서 고른 것이므로 최소 한 대는 반드시 겹치고, 그 겹친 노드가 최신 값을 갖고 있습니다.

여기에 더해, 노드들이 서로 데이터를 주고받으며 손상된 사본을 스스로 복구합니다. 6장이 따로따로 설명한 데이터 다중화, 정족수, 장애 복구가 한 서비스 안에 다 들어 있는 셈입니다.

그래서 단점은

RDS보다 비쌉니다. 대신 용량 관리, 장애 조치, 복제본 부하 분산을 전부 알아서 해줍니다. 관리할 사람이 없는 팀일수록 값어치를 합니다.

06 ElastiCache — 관리형 캐시

RDS가 관계형 DB를 대신 관리해 주는 서비스라면, ElastiCache는 Redis나 Memcached를 대신 관리해 주는 서비스입니다. 운영체제 패치, 설정, 모니터링, 장애 복구, 백업을 AWS가 맡습니다.

어디에 쓰나

- DB 앞의 캐시 — 애플리케이션이 먼저 캐시를 보고, 없으면 DB에서 읽어 캐시에 넣습니다. DB 부하가 크게 줄어듭니다.
- 세션 저장소 — 로그인 상태를 서버 바깥에 두면, 사용자가 어느 서버로 가도 로그인 상태가 유지됩니다. 서버를 늘리고 줄이기 쉬워집니다.
- 실시간 랭킹 — Redis의 Sorted Set은 값을 넣는 즉시 순위가 매겨집니다. 게임 리더보드 같은 데 씁니다.

공짜가 아닙니다

RDS는 기존 DB를 대신 굴려주는 것이라 앱을 거의 안 고쳐도 되지만, 캐시를 도입하려면 애플리케이션 코드를 상당히 손봐야 합니다. 어디서 캐시를 보고, 언제 갱신하고, 언제 지울지를 전부 직접 정해야 하기 때문입니다.

Redis와 Memcached — 6장의 복제 vs 파티션이 그대로 나타납니다

	Redis	Memcached
복제 · 고가용성	Multi-AZ + 자동 장애 조치, 읽기 복제본	없음
샤딩	클러스터 모드를 켜면 가능	다중 노드로 데이터 분할
영속성	AOF 로그로 디스크에 기록 가능	없음 — 꺼지면 사라짐
자료구조	Set, Sorted Set 등 풍부함	단순 키-값
기타	백업 · 복원 지원	멀티스레드 구조

흔히 "Redis는 복제, Memcached는 샤딩"으로 대비시키는데, 정확히는 **Redis는 둘 다 되고 Memcached는 샤딩만 됩니다**. Redis 클러스터 모드를 켜면 데이터를 여러 샤드로 쪼갬 뒤 각 샤드를 다시 복제하는데, 이게 바로 6장에서 말한 "파티션으로 쪼갬 각 조각을 다시 복제한다"는 구성 그 자체입니다.

덧붙여, Redis 클러스터의 노드들은 서로 상태를 주고받으며 어느 노드가 죽었는지 퍼뜨립니다. 6장의 **가십(Gossip) 프로토콜**이 실제로 쓰이는 자리입니다.

캐시 전략 두 가지

- **Lazy Loading** — 읽을 때 캐시에 없으면 DB에서 가져와 캐시에 넣습니다. 구현이 쉽지만, DB가 바뀌어도 캐시는 옛날 값을 들고 있을 수 있습니다.
- **Write Through** — DB에 쓸 때 캐시도 같이 갱신합니다. 옛날 값이 남지 않지만, 쓸 때마다 일이 늘고 아무도 안 읽을 데이터 까지 캐시에 쌓입니다.

07 6장 개념 ↔ AWS 서비스 한눈에

6장 개념	AWS	어떻게 대응되나
분산 키-값 저장소	ElastiCache (Redis)	Redis 자체가 분산 인메모리 키-값 저장소입니다. 6장은 이런 것을 어떻게 만드는지를 다룹니다.
데이터 파티션	Redis Cluster	키를 고정된 슬롯에 배정하고, 슬롯을 노드에 나눠 맡깁니다. 노드가 늘어도 슬롯 일부만 옮기면 되는데, 5장의 가상 노드와 같은 발상입니다.
데이터 다중화	RDS Multi-AZ Aurora 6벌 저장	같은 데이터를 여러 벌 두어 장애에 대비합니다. 서로 다른 AZ에 두는 것은 책의 "데이터 센터 장애 처리"에 해당합니다.
정족수 (N·W·R)	Aurora	$N=6, W=4, R=3$. $W+R>N$ 조건이 실제 제품에서 이 숫자로 구현돼 있습니다.
일관성 모델	Multi-AZ ↔ Read Replica	동기 복제는 강한 일관성, 비동기 복제는 결과적 일관성입니다. 복제 지연이 결과적 일관성의 실제 모습입니다.
장애 감지 (가십)	ElastiCache Redis Cluster	노드들이 서로 상태를 주고받으며 죽은 노드를 알아냅니다.
장애 복구	Aurora	노드끼리 복제하며 손상된 사본을 스스로 복구합니다.
고가용성	ELB + ASG, 다중 AZ	한 대가 죽어도 로드밸런서가 빼고, ASG가 새로 채웁니다.

아직 안 배운 것 하나

6장이 설계하는 키-값 저장소의 원형은 아마존의 **Dynamo** 논문입니다. AWS의 DynamoDB가 그 후손입니다. SAA 강의에서는 한참 뒤 서버리스 파트에서 다루기 때문에 이 문서에서는 다루지 않았습니다.

08 더 보려면

AWS 공식 문서

ELB 전반 <https://docs.aws.amazon.com/elasticloadbalancing/>

RDS Multi-AZ <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html>

RDS Read Replica https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html

Aurora <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/>

ElastiCache <https://docs.aws.amazon.com/AmazonElastiCache/>

대규모 시스템 설계 스터디 4주차 · 6-7장 발표 부록 | 송승환 | 2026.09

AWS 서비스의 사양과 한도는 자주 바뀝니다. 실제 설계에 쓰기 전에는 위 공식 문서에서 최신 값을 확인하세요.